

CS3213: Foundations of Software Engineering

Statistical Fault Localization

Ruling Out Program States

```
public static int middle(int x, int y, int z) {  
    if (y < z) {  
        if (x < y) {  
            return y;  
        } else if (x < z) {  
            return y;  
        }  
    } else {  
        if (x > y) {  
            return y;  
        } else if (x > z) {  
            return x;  
        }  
    }  
    return z;  
}
```

```
int m = middle(2, 1, 3);  
System.out.println(m);
```

A *slice* identifies the subset of the program that could have influenced the variable/returned value



Statistical Fault Localization

- Question: how to automatically localize faults?
- ***Statistical debugging***: use statistical reasoning for debugging
- ***Statistical fault localization***: utilize multiple executions for localizing faults (a category of statistical debugging)
- Observation: whether a program passes or fails likely correlates with events that precede it
 - Correlation does not imply causation
 - Can help locate faults

Example

abc



```
public static String removeHtmlMarkup(String html) {  
    boolean tag = false;  
    boolean quote = false;  
    String out = "";  
    for (char c : html.toCharArray()) {  
        if (c == '<' && !quote) {  
            tag = true;  
        } else if (c == '>' && !quote) {  
            tag = false;  
        } else if (c == '"' || c == '\'' && tag) {  
            quote = !quote;  
        } else if (!tag) {  
            out += c;  
        }  
    }  
    return out;  
}
```

→ abc



For this test case, the result was correct, so we have some **confidence** that the executed code (in green) **works correctly**

Example

"abc"



```
public static String removeHtmlMarkup(String html) {  
    boolean tag = false;  
    boolean quote = false;  
    String out = "";  
    for (char c : html.toCharArray()) {  
        if (c == '<' && !quote) {  
            tag = true;  
        } else if (c == '>' && !quote) {  
            tag = false;  
        } else if (c == '"' || c == '\'' && tag) {  
            quote = !quote;  
        } else if (!tag) {  
            out += c;  
        }  
    }  
    return out;  
}
```

→ abc ❌

Some of the logic involved in executing the test is buggy, so we need to consider the relevant lines as **potential culprits**

Example

Line	abc ✓	abc ✓	"abc" ✗
1 (boolean tag = false;)	X	X	X
2	X	X	X
3	X	X	X
4	X	X	X
5	X	X	X
6 (tag = true;)	-	X	-
7	X	X	X
8 (tag = false;)	-	X	-
9	X	X	X
10 (quote = !quote;)	-	-	X
11	X	X	X
12	X	X	X
15	X	X	X

Example

Line	abc ✓	abc ✓	"abc" ✗
1 (boolean tag = false;)	X	X	X
2	X	X	X
3	X	X	X
4	X	X	X
5	X	X	X
6 (tag = true;)	-	X	-
7	X	X	X
8 (tag = false;)			
9			
10 (quote = !quote;)			
11	X	X	X
12	X	X	X
15	X	X	X

Not executed by the failing test, so definitely not related to the defect

Example

Line	abc ✓	abc ✓	"abc" X
1 (boolean tag = false;)	X	X	X
2	X	X	X
3	X	X	
4	X	X	
5	X	X	
6 (tag = true;)	-	X	
7	X	X	^
8 (tag = false;)	-	X	-
9	X	X	X
10 (quote = !quote;)	-	-	X
11	X	X	X
12	X	X	X
15	X	X	X

Only executed by the failing test, so potentially connected with the defect

How can we use this intuition to help us find the defect?



Tarantula

- One of the most widely known statistical debugging techniques
- Instance of a spectrum-based fault localization technique
 - Program spectra: execution profiles such as statement coverage
- **Intuition:** program elements that are executed by a failing test are more likely to be faulty (i.e., ***suspicious***) than those primarily executed by passing test cases
 - Ranks statements for further analysis



Tarantula

$$Suspiciousness(line) = \frac{\frac{failed(line)}{total\ failed}}{\frac{failed(line)}{total\ failed} + \frac{passed(line)}{total\ passed}}$$

Assign zero if any denominators evaluate to zero

Tarantula: Suspiciousness Score (1/3)

Line	abc ✓	abc ✓	"abc" ✗
1 (boolean tag = false;)	X	X	X
2	X	X	X
3	X	X	X
4	X	X	X
5	X	X	X
6 (tag = true;)	-	X	-
7	X	X	X
8 (tag			
9			
10 (qu			
11			
12			
15			

$$\text{Suspiciousness}(6) = \frac{\frac{\text{failed}(6)}{\text{total failed}}}{\frac{\text{failed}(6)}{\text{total failed}} + \frac{\text{passed}(6)}{\text{total passed}}} = \frac{\frac{0}{1}}{\frac{0}{1} + \frac{1}{2}} = 0$$

Given that no failing test case exercised this line, it is **not suspicious**

Tarantula: Suspiciousness Score (2/3)

Line	abc ✓	abc ✓	"abc" ✗
1 (boolean tag = false;)	X	X	X
2	X	X	X
3	X	X	X
4	X	X	X
$\text{Suspiciousness}(10) = \frac{\frac{\text{failed}(10)}{\text{total failed}}}{\frac{\text{failed}(10)}{\text{total failed}} + \frac{\text{passed}(10)}{\text{total passed}}} = \frac{\frac{1}{1}}{\frac{1}{1} + \frac{0}{2}} = 1$			X
8 (tag = false;)	-	X	-
9	X	X	X
10 (quote = !quote;)	-	-	X
11	X	X	X
12	>		
15	>		

Highly suspicious line, as it was covered only in failing tests

Tarantula: Suspiciousness Score (3/3)

Line	abc ✓	abc ✓	"abc" ✗
1 (boolean tag = false;)	X	X	X
2	X	X	X
3	$\text{Suspiciousness}(7) = \frac{\frac{\text{failed}(7)}{\text{total failed}}}{\frac{\text{failed}(7)}{\text{total failed}} + \frac{\text{passed}(7)}{\text{total passed}}} = \frac{\frac{1}{1}}{\frac{1}{1} + \frac{2}{2}} = 0.5$		
4			
5			
6 (tag = true;)			
7	X	X	X
8 (tag = false;)	-	X	-
9	X	X	X
10 (quote = !quote;)	-	Medium suspiciousness , as this line was covered both by the passing and failing test(s)	
11	X		
12	X		
15	X		



Tarantula: Hue

- Goal: visualize the suspiciousness score in the editor by assigning a color (or hue) to statements depending how *suspicious* they are
 - From red to yellow to green
 - Red are suspicious
 - Green are safe
 - Yellow are somewhere in between

Tarantula: Hue

- Hue of 0.0 stands for red
- Hue of 1.0 stands for green
- Hue of 0.5 stands for equal fractions of red and green, yielding yellow

$$\text{color hue (line)} = \text{low color (red)} + \frac{\%passed(line)}{\%passed(line) + \%passed(line)} \times \text{color range}$$

Example

"abc"



```
public static String removeHtmlMarkup(String html) {  
    boolean tag = false;  
    boolean quote = false;  
    String out = "";  
    for (char c : html.toCharArray()) {  
        if (c == '<' && !quote) {  
            tag = true;  
        } else if (c == '>' && !quote) {  
            tag = false;  
        } else if (c == '"' || c == '\'' && tag) {  
            quote = !quote;  
        } else if (!tag) {  
            out += c;  
        }  
    }  
    return out;  
}
```

The line with the highest suspiciousness is related to the defect

→ abc X

Abstract—Software fault localization, the act of identifying the locations of faults in a program, is widely recognized to be one of the most tedious, time consuming, and expensive – yet equally critical – activities in program debugging. Due to the increasing scale and complexity of software today, manually locating faults when failures occur is rapidly becoming infeasible, and consequently, there is a strong demand for techniques that can guide software developers to the locations of faults in a program with minimal human intervention. This demand in turn has fueled the proposal and development of a broad spectrum of fault localization techniques, each of which aims to streamline the fault localization process and make it more effective by attacking the problem in a unique way. In this article, we catalog and provide a comprehensive overview of such techniques and discuss key issues and concerns that are pertinent to software fault localization as a whole.

Index Terms—Software fault localization, program debugging, software testing, execution trace, suspicious code, survey

1 INTRODUCTION

SOFTWARE is fundamental to our lives today, and with its ever-increasing usage and adoption, its influence is practically ubiquitous. In fact, at present, software is not just employed in, but is critical to, many security and safety-critical systems in industries such as medicine, aerospace, and nuclear energy. Not surprisingly, this trend has been accompanied by a drastic increase in the scale and complexity of software. Unfortunately, this has also resulted in more software bugs, which often lead to execution failures with huge losses [26], [27], [36]. Furthermore, software faults in safety-critical systems have significant ramifications, including not only financial loss, but also potential loss of life, which is an alarming prospect [36]. A 2002 report from the National Institute of Standards and Technology (NIST) [34] indicated that software errors are estimated to cost the U.S. economy \$59.5 billion annually (0.6 percent of the GDP); the cost has undoubtedly grown since then. Over half the cost of fixing or responding to these bugs is passed on to software users, while software developers and vendors absorb the rest.

Even when faults in software are discovered due to erroneous behavior or some other manifestation of the faults¹, finding and fixing them is an entirely different matter. Fault

localization, which focuses on the former, i.e., identifying the locations of faults, has historically been a manual task that has been recognized to be time consuming and tedious as well as prohibitively expensive [347], given the size and complexity of large-scale software systems today. Furthermore, manual fault localization relies heavily on the software developer's experience, judgment, and intuition to identify and prioritize code that is likely to be faulty. These limitations have led to a surge of interest in developing techniques that can partially or fully automate the localization of faults in software while reducing human input. Though some techniques are similar and some very different (in terms of the type of data consumed, the program components focused on, comparative effectiveness and efficiency, etc.), they each try to attack the problem of fault localization from a unique perspective, and typically offer both advantages and disadvantages relative to one another. With many techniques already in existence and others continually being proposed, as well as with advances being made both from a theoretical and practical perspective, it is important to catalog and overview current techniques in fault localization in order to offer a comprehensive resource for those already in the area as well as those interested in making contributions to it.

Fault Localization

Coefficient	Algebraic Form	Coefficient	Algebraic Form
1 Braun-Banquet	$\frac{N_{CF}}{\max(N_{CF}+N_{CS}, N_{CF}+N_{UF})}$	17 Harmonic Mean	$\frac{(N_{CF} \times N_{US} - N_{UF} \times N_{CS})((N_{CF}+N_{CS}) \times (N_{US}+N_{UF}) + (N_{CF}+N_{UF}) \times (N_{CS}+N_{US}))}{(N_{CF}+N_{CS}) \times (N_{US}+N_{UF}) \times (N_{CF}+N_{UF}) \times (N_{CS}+N_{US})}$
2 Dennis	$\frac{(N_{CF} \times N_{US}) - (N_{CS} \times N_{UF})}{\sqrt{n \times (N_{CF}+N_{CS}) \times (N_{CF}+N_{UF})}}$	18 Rogot2	$\frac{1}{4} \left(\frac{N_{CF}}{N_{CF}+N_{CS}} + \frac{N_{CF}}{N_{CF}+N_{UF}} + \frac{N_{US}}{N_{US}+N_{CS}} + \frac{N_{US}}{N_{US}+N_{UF}} \right)$
3 Mountford	$\frac{N_{CF}}{0.5 \times ((N_{CF} \times N_{CS}) + (N_{CF} \times N_{UF})) + (N_{CS} \times N_{UF})}$	19 Simple Matching	$\frac{N_{CF}+N_{US}}{N_{CF}+N_{CS}+N_{US}+N_{UF}}$
4 Fossum	$\frac{n \times (N_{CF} - 0.5)^2}{(N_{CF}+N_{CS}) \times (N_{CF}+N_{UF})}$	20 Rogers & Tanimoto	$\frac{N_{CF}+N_{US}}{N_{CF}+N_{US}+2(N_{UF}+N_{CS})}$
5 Pearson	$\frac{n \times ((N_{CF} \times N_{US}) - (N_{CS} \times N_{UF}))^2}{N_{CF} \times N_{US} \times N_{CS} \times N_{UF}}$	21 Hamming	$N_{CF} + N_{US}$
6 Gower	$\frac{N_{CF}+N_{US}}{\sqrt{N_{CF} \times N_{CS} \times N_{UF} \times N_{US}}}$	22 Hamann	$\frac{N_{CF}+N_{US}-N_{UF}-N_{CS}}{N_{CF}+N_{UF}+N_{CS}+N_{US}}$
7 Michael	$\frac{4 \times ((N_{CF} \times N_{US}) - (N_{CS} \times N_{UF}))}{(N_{CF}+N_{US})^2 + (N_{CS}+N_{UF})^2}$	23 Sokal	$\frac{2(N_{CF}+N_{US})}{2(N_{CF}+N_{US})+N_{UF}+N_{CS}}$
8 Pier...			
9 Bar...			
10 Tar...			
11 Am...			
12 Phi (Geometric Mean)	$\frac{N_{CF} \times N_{US} - N_{UF} \times N_{CS}}{\sqrt{(N_{CF}+N_{CS}) \times (N_{CF}+N_{UF}) \times (N_{CS}+N_{US}) \times (N_{UF}+N_{US})}}$	28 Dice	$\frac{2N_{CF}}{N_{CF}+N_{UF}+N_{CS}}$
13 Arithmetic Mean	$\frac{2(N_{CF} \times N_{US} - N_{UF} \times N_{CS})}{(N_{CF}+N_{CS}) \times (N_{US}+N_{UF}) + (N_{CF}+N_{UF}) \times (N_{CS}+N_{US})}$	29 Goodman	$\frac{2N_{CF} - N_{UF} - N_{CS}}{2N_{CF}+N_{UF}+N_{CS}}$
14 Cohen	$\frac{2(N_{CF} \times N_{US} - N_{UF} \times N_{CS})}{(N_{CF}+N_{CS}) \times (N_{US}+N_{CS}) + (N_{CF}+N_{UF}) \times (N_{UF}+N_{US})}$	30 Jaccard	$\frac{N_{CF}}{N_{CF}+N_{UF}+N_{CS}}$
15 Fleiss	$\frac{4(N_{CF} \times N_{US} - N_{UF} \times N_{CS}) - (N_{UF} - N_{CS})^2}{(2N_{CF}+N_{UF}+N_{CS}) + (2N_{US}+N_{UF}+N_{CS})}$	31 Sorensen-Dice	$\frac{2N_{CF}}{2N_{CF}+N_{UF}+N_{CS}}$
16 Zoltar	$\frac{N_{CF}}{N_{CF}+N_{UF}+N_{CS} + \frac{10000 \times N_{UF} \times N_{CS}}{N_{CF}}}$		

Over the last decades, many statistical debugging and fault localization techniques have been proposed!



Statistical Debugging Tools Adoption

- Limited tool support in the form of IDE integrations
- Research has shown that such approaches provide only (limited) support to experts
- Useful as ingredients in other approaches (e.g., automatic program repair)



Takeaways and Summary

- Statistical fault location prioritizes locations in the code to analyze
- Tarantula
- Fault localization and adoption